

CIFAR-10 图像分类

代码结构

1. 环境导入与配置

```
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
import torchvision
import torchvision.transforms as transforms
import matplotlib.pyplot as plt
import matplotlib

matplotlib.rcParams['font.family'] = 'SimHei'
```

- 导入环境
- 配置中文字体

2. 加载并预处理CIFAR-10数据集

```
transform = transforms.Compose(
    [transforms.ToTensor(),
     transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))])

batch_size = 64
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

trainset = torchvision.datasets.CIFAR10(root='./data', train=True, download=False,
transform=transform)
trainloader = torch.utils.data.DataLoader(trainset, batch_size=batch_size,
shuffle=True)

testset = torchvision.datasets.CIFAR10(root='./data', train=False, download=True,
transform=transform)
testloader = torch.utils.data.DataLoader(testset, batch_size=batch_size,
shuffle=False)

classes = ('plane', 'car', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse', 'ship',
'truck')
```

3. 构建CNN模型

```
class SimpleCNN(nn.Module):
    def __init__(self):
```

```
super().__init__()
# --- 在这里定义你的网络层 ---
self.conv1 = nn.Conv2d(3, 32, kernel_size=3, padding=1)
self.conv2 = nn.Conv2d(32, 64, kernel_size=3, padding=1)
self.conv3 = nn.Conv2d(64, 128, kernel_size=3, padding=1)
self.pool = nn.MaxPool2d(2, 2)
self.fc1 = nn.Linear(128 * 4 * 4, 512)
self.fc2 = nn.Linear(512, 10)
self.dropout = nn.Dropout(0.5)

def forward(self, x):
    # --- 在这里定义前向传播逻辑 ---
    x = self.pool(F.relu(self.conv1(x)))
    x = self.pool(F.relu(self.conv2(x)))
    x = self.pool(F.relu(self.conv3(x)))
    x = torch.flatten(x, 1) # 展平张量
    x = F.relu(self.fc1(x))
    x = self.dropout(x)
    x = self.fc2(x)
    return x

net = SimpleCNN().to(device)
```

4. 模型训练与评估

```
epoch_num = 10

train_loss_values = []
train_acc_values = []
test_acc_values = []

for epoch in range(epoch_num): # 训练10个epoch
    running_loss = 0.0
    correct = 0
    total = 0

    net.train() # 设置为训练模式
    for i, data in enumerate(trainloader, 0):
        inputs, labels = data
        inputs, labels = inputs.to(device), labels.to(device)

        # 梯度清零
        optimizer.zero_grad()

        # 前向传播 + 反向传播 + 优化
        outputs = net(inputs)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

    # 统计训练准确率
    _, predicted = torch.max(outputs.data, 1)
```

```

        total += labels.size(0)
        correct += (predicted == labels).sum().item()

    # 累加损失
    running_loss += loss.item()

# 计算并记录训练损失和准确率
train_loss = running_loss / len(trainloader)
train_acc = 100 * correct / total
train_loss_values.append(train_loss)
train_acc_values.append(train_acc)

# 在测试集上评估模型
net.eval() # 设置为评估模式
correct = 0
total = 0
with torch.no_grad():
    for data in testloader:
        images, labels = data
        images, labels = images.to(device), labels.to(device)
        outputs = net(images)
        _, predicted = torch.max(outputs.data, 1)
        total += labels.size(0)
        correct += (predicted == labels).sum().item()

test_acc = 100 * correct / total
test_acc_values.append(test_acc)

print(f'Epoch {epoch + 1}, 训练损失: {train_loss:.4f}, 训练准确率: {train_acc:.2f}%, 测试准确率: {test_acc:.2f}%')

print('训练完成')

```

- 每次处理64个样本
- 运用Adam优化器
- 交叉熵损失

5. 结果可视化

```

plt.figure(figsize=(12, 5))

# 绘制损失曲线
plt.subplot(1, 2, 1)
plt.plot(range(1, epoch_num + 1), train_loss_values, marker='o', color='blue')
plt.title('训练过程中的损失变化')
plt.xlabel('Epoch')
plt.ylabel('损失值')
plt.grid(True)

# 绘制准确率曲线
plt.subplot(1, 2, 2)

```

```
plt.plot(range(1, epoch_num + 1), train_acc_values, marker='o', color='blue',
label='训练准确率')
plt.plot(range(1, epoch_num + 1), test_acc_values, marker='s', color='red',
label='测试准确率')
plt.title('训练过程中的准确率变化')
plt.xlabel('Epoch')
plt.ylabel('准确率 (%)')
plt.grid(True)
plt.legend()

# 保存图像
plt.tight_layout()
plt.show()

# 显示模型在各个类别上的准确率
class_correct = list(0. for i in range(10))
class_total = list(0. for i in range(10))
with torch.no_grad():
    for data in testloader:
        images, labels = data
        images, labels = images.to(device), labels.to(device)
        outputs = net(images)
        _, predicted = torch.max(outputs, 1)
        c = (predicted == labels).squeeze()
        for i in range(labels.size(0)):
            label = labels[i]
            if len(c.shape) > 0:
                class_correct[label] += c[i].item()
            else:
                class_correct[label] += c.item()
            class_total[label] += 1

print('\n各类别准确率:')
for i in range(10):
    print(f'{classes[i]}的准确率: {100 * class_correct[i] / class_total[i]:.2f}%')
```

- 损失曲线
- 准确率比较

预期输出

- 数据加载信息
- 每轮epoch
- 训练损失曲线，准确率对比曲线

一些尝试（相关截图见image文件夹）

- 学习率：经尝试，学习率在0.0005-0.0015之间效果最佳，最后使用0.001
- 增加卷积层，池化层：收敛速度减慢，过拟合程度增大，准确率稍稍下降